Московский государственный университет имени М. В. Ломоносова
механико-математический факультет
Практикум на ЭВМ

Отчет №1 студента 405
группы Филатова Никиты
Сергеевича

Вариант №21 : "Решение краевой задачи методом конечных разностей."

Москва, 2025

Содержание

1. Общая постановка задачи 2
2. Метод решения 2
3. Решение системы нелинейных уравнений методом Ньютона 3
4. Результаты и тестирование 5
5. Физическая интерпретация 9
6. Выводы 9
7. Список литературы 10
8. Приложение. Код программы 11

1 Постановка задачи

Аппроксимировать со вторым порядком следующую задачу с помощью метода конечных разностей и найти решение полученной системы алгебраических уравнений при различных h, f и α:

$$\begin{cases} -u'' + \alpha \dfrac{uv}{u^2+v^2} = f(x), \\ v' - \alpha \dfrac{uv}{u^2+v^2} = g(x), \end{cases} \qquad x \in [0,1]$$

с краевыми условиями:

$$u(0) = 0, \quad u'(1) = 0, \quad v(1) = 0$$

2 Метод решения

Для решения используется метод конечных разностей второго порядка точности. Отрезок $[0,1]$ разбивается на N равных частей длиной $h = 1/N$.

Аппроксимации производных:

$$u'(x_i) \approx \frac{u_{i+1} - u_{i-1}}{2h}, \quad v'(x_i) \approx \frac{v_{i+1} - v_{i-1}}{2h}$$

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}, \quad v''(x_i) \approx \frac{v_{i-1} - 2v_i + v_{i+1}}{h^2}$$

Аппроксимация условия $u'(1) = 0$:

$$u'_N \approx \frac{3u_N - 4u_{N-1} + u_{N-2}}{2h} = 0 \Rightarrow 3u_N - 4u_{N-1} + u_{N-2} = 0$$

Аппроксимация $v'(0) = g(0)$:

$$v'_0 \approx \frac{-3v_0 + 4v_1 - v_2}{2h} = g(0) \Rightarrow -3v_0 + 4v_1 - v_2 - 2hg(0) = 0$$

Далее подставляем эти аппроксимации в систему. Но перед этим заметим, что особенность этой задачи заключается в том, что может обнулиться знаменатель $u^2 + v^2$, поэтому я прибавил $\varepsilon = 10^{-8}$. Тогда обозначим нелинейный член за $Q(u,v) = \frac{uv}{u^2+v^2+\varepsilon}$

После применения метода конечных разностей второго порядка, исходная система дифференциальных уравнений преобразуется в следующую систему нелинейных алгебраических уравнений:

Для внутренних узлов $i = 1, 2, \ldots, N-1$:

$$-\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + \alpha Q(u_i, v_i) = f(x_i)$$
$$\frac{v_{i+1} - v_{i-1}}{2h} - \alpha Q(u_i, v_i) = g(x_i)$$

Умножая на $-h^2$ в уравнении для u и на $2h$ в уравнении для v и перенося все члены в левую сторону, получаем:

$$u_{i-1} - 2u_i + u_{i+1} - h^2 \alpha Q(u_i, v_i) + h^2 f(x_i) = 0 \tag{1}$$
$$v_{i+1} - v_{i-1} - 2h\alpha Q(u_i, v_i) - 2hg(x_i) = 0 \tag{2}$$

Граничные условия:

$$u_0 = 0 \tag{3}$$
$$3u_N - 4u_{N-1} + u_{N-2} = 0 \tag{4}$$
$$-3v_0 + 4v_1 - v_2 - 2hg(0) = 0 \tag{5}$$
$$v_N = 0 \tag{6}$$
$$\tag{7}$$

Получаем систему из $2N$ нелинейных уравнений относительно $2N$ неизвестных:

- $u_1, u_2, \ldots, u_N$ (N неизвестных)
- $v_0, v_1, \ldots, v_{N-1}$ (N неизвестных)

3 Решение системы нелинейных уравнений методом Ньютона

После аппроксимации исходной системы дифференциальных уравнений методом конечных разностей мы получаем систему нелинейных алгебраических уравнений (СНАУ). Эту систему можно представить в компактной векторной форме:

$$F(X) = 0, \tag{8}$$

где X – вектор неизвестных.

В нашей задаче вектор неизвестных X имеет размерность $2N$ и состоит из сеточных значений функций u и v:

$$X = [u_1, u_2, \ldots, u_N, v_0, v_1, \ldots, v_{N-1}]^T. \tag{9}$$

Вектор $F(X)$ также имеет размерность $2N$, и его компоненты представляют собой левые части конечно-разностных уравнений для u и v во всех узлах сетки.

Для решения системы $F(X) = 0$ применяется итерационный метод Ньютона (также известный как метод Ньютона-Рафсона). Идея метода заключается в последовательной линеаризации системы в окрестности текущего приближения.

Пусть X_k – приближение к решению на k-й итерации. Тогда следующее приближение X_{k+1} ищется в виде $X_{k+1} = X_k + \Delta X_k$, где поправка ΔX_k находится из решения системы линейных алгебраических уравнений (СЛАУ):

$$J(X_k)\Delta X_k = -F(X_k). \tag{10}$$

Здесь $J(X_k)$ – это матрица Якоби системы, вычисленная в точке X_k. Элементы матрицы Якоби определяются как частные производные компонент вектор-функции F по компонентам вектора неизвестных X:

$$J_{ij}(X) = \frac{\partial F_i}{\partial X_j}. \tag{11}$$

Структура матрицы Якоби

Матрица Якоби для нашей системы имеет блочную структуру 2×2, отражающую связь между уравнениями для u и v:

$$J = \begin{bmatrix} \dfrac{\partial F_u}{\partial u} & \dfrac{\partial F_u}{\partial v} \\ \dfrac{\partial F_v}{\partial u} & \dfrac{\partial F_v}{\partial v} \end{bmatrix}, \tag{12}$$

где F_u и F_v – части вектора, относящиеся к уравнениям для u и v соответственно, а u и v – соответствующие части вектора неизвестных X.

Элементы матрицы Якоби

Блок $\dfrac{\partial F_u}{\partial u}$ (размер $N \times N$):

- Для внутренних узлов $i = 1, 2, \ldots, N-1$:

$$\frac{\partial F_{u,i}}{\partial u_{i-1}} = 1$$
$$\frac{\partial F_{u,i}}{\partial u_i} = -2 - h^2 \cdot \frac{\partial Q}{\partial u}(u_i, v_i)$$
$$\frac{\partial F_{u,i}}{\partial u_{i+1}} = 1$$

- Для граничного условия $u'(1) = 0$:

$$\frac{\partial F_{u,N}}{\partial u_{N-2}} = 1$$
$$\frac{\partial F_{u,N}}{\partial u_{N-1}} = -4$$
$$\frac{\partial F_{u,N}}{\partial u_N} = 3$$

Блок $\dfrac{\partial F_u}{\partial v}$ (размер $N \times N$):

- Для внутренних узлов $i = 1, 2, \ldots, N-1$:

$$\frac{\partial F_{u,i}}{\partial v_i} = -h^2 \cdot \frac{\partial Q}{\partial v}(u_i, v_i)$$

Блок $\dfrac{\partial F_v}{\partial u}$ (размер $N \times N$):

- Для внутренних узлов $i = 1, 2, \ldots, N-1$:

$$\frac{\partial F_{v,i}}{\partial u_i} = -2h \cdot \frac{\partial Q}{\partial u}(u_i, v_i)$$

Блок $\dfrac{\partial F_v}{\partial v}$ (размер $N \times N$):

- Для граничного условия $v'(0) = g(0)$:

$$\frac{\partial F_{v,0}}{\partial v_0} = -3$$
$$\frac{\partial F_{v,0}}{\partial v_1} = 4$$
$$\frac{\partial F_{v,0}}{\partial v_2} = -1$$

- Для внутренних узлов $i = 1, 2, \ldots, N-1$:

$$\frac{\partial F_{v,i}}{\partial v_{i-1}} = -1$$
$$\frac{\partial F_{v,i}}{\partial v_i} = -2h \cdot \frac{\partial Q}{\partial v}(u_i, v_i)$$
$$\frac{\partial F_{v,i}}{\partial v_{i+1}} = 1$$

Частные производные нелинейного члена
Нелинейный член $Q(u, v)$ и его производные:

$$Q(u, v) = \alpha \frac{uv}{u^2 + v^2 + \varepsilon} \tag{8}$$
$$\frac{\partial Q}{\partial u} = \alpha \frac{v(v^2 + \varepsilon - u^2)}{(u^2 + v^2 + \varepsilon)^2} \tag{9}$$
$$\frac{\partial Q}{\partial v} = \alpha \frac{u(u^2 + \varepsilon - v^2)}{(u^2 + v^2 + \varepsilon)^2} \tag{10}$$

Начальное приближение
В качестве начального приближения X_0 используется решение линейной задачи ($\alpha = 0$):

- Для $u(x)$ решается краевая задача:

$$\begin{cases} -u'' = f(x) \\ u(0) = 0, \quad u'(1) = 0 \end{cases}$$

- Для $v(x)$ решается задача Коши:

$$\begin{cases} v' = g(x) \\ v(1) = 0 \end{cases}$$

Алгоритм метода Ньютона
Итерационный процесс решения СНАУ выглядит следующим образом:

1. **Инициализация.** Задать начальное приближение X_0 как решение линейной задачи. Установить счетчик итераций $k = 0$ и задать критерий сходимости $\varepsilon = 10^{-8}$.

2. **Итерационный цикл.** Для $k = 0, 1, 2, \ldots$ до достижения сходимости:

   (a) Вычислить вектор $F(X_k)$, используя текущее приближение.

   (b) Вычислить матрицу Якоби $J(X_k)$.

   (c) Решить СЛАУ (10) для нахождения вектора поправки ΔX_k.

   (d) Обновить вектор решения: $X_{k+1} = X_k + \Delta X_k$.

3. **Проверка сходимости.** Проверить условие остановки. Итерации прекращаются, если норма вектора поправки становится достаточно малой:

$$\|\Delta X_k\| < \varepsilon, \tag{13}$$

   где $\|\cdot\|$ – евклидова (L2) норма, а $\varepsilon = 10^{-8}$ – заданная малая величина. Если сходимость не достигнута за максимальное число итераций (100), процесс прерывается.

После завершения итерационного процесса вектор X_{k+1} принимается за искомое решение системы.

4 Результаты и тестирование.

Пример 1 С такими условиями $\alpha = 0$, $f(x) = 0$, $g(x) = 0$ получаем следующую систему:

$$\begin{cases} -u'' = 0 \\ v' = 0 \end{cases}$$

Общее решение: $u(x) = C_1 x + C_2$, $v(x) = C_3$

Решение с учетом краевых условий: $u(x) = 0$, $v(x) = 0$ Как мы видим, наше решение совпало с точным решением

Погрешность: $0.00e + 00$

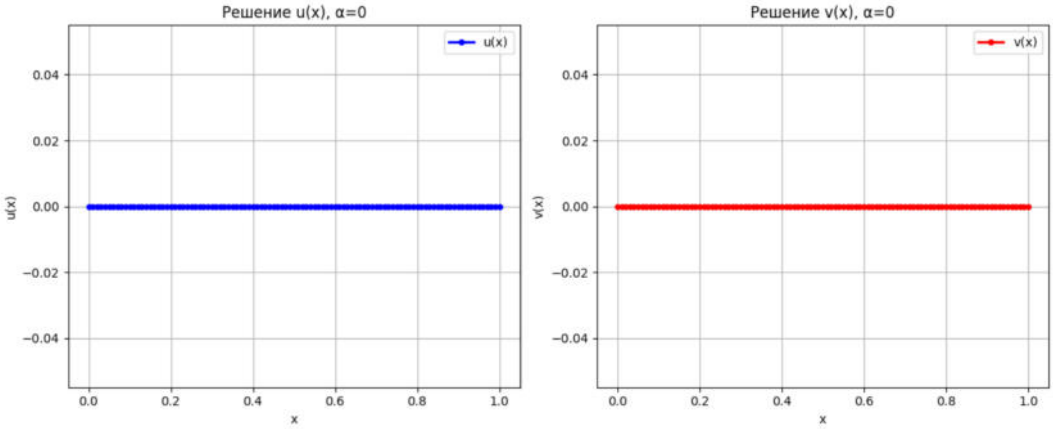

Рис. 1: $N = 100$

Пример 2 Теперь рассмотрим функции $u(x) = x(1 - x/2)$ и $v(x) = x(1 - x)$, которые удовлетворяют краевым условиям и обнуляют знаменатель в 0 и 1. При $\alpha = 1$ вычислим правые части.

Погрешность: $4.90e - 14$

Как мы видим, не смотря на обнуление знаменателя, все хорошо, потому что я добавлял константу ε

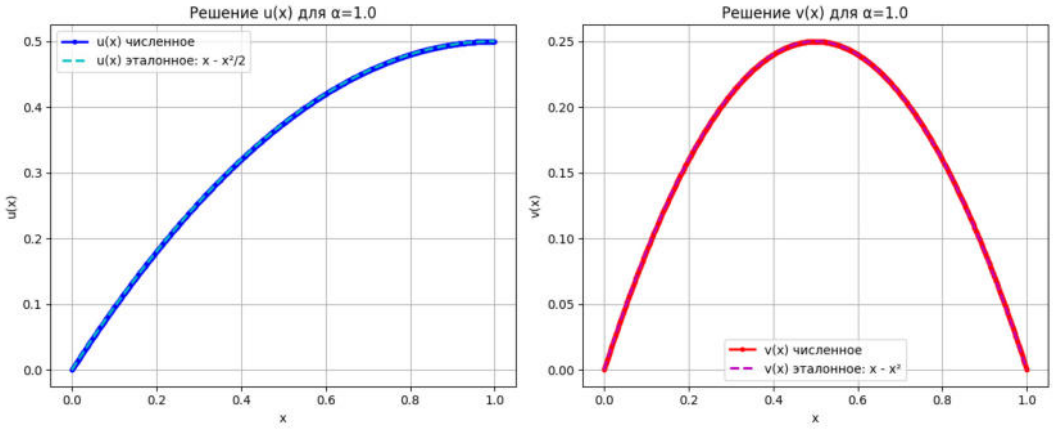

Рис. 2: $N = 1000$

Пример 3 Теперь рассмотрим функции $u(x) = sin(\pi/2 * x)$ и $v(x) = sin(\pi * x)$, которые удовлетворяют краевым условиям. При $\alpha = 1$ вычислим правые части.

Погрешность: $1.60e - 06$

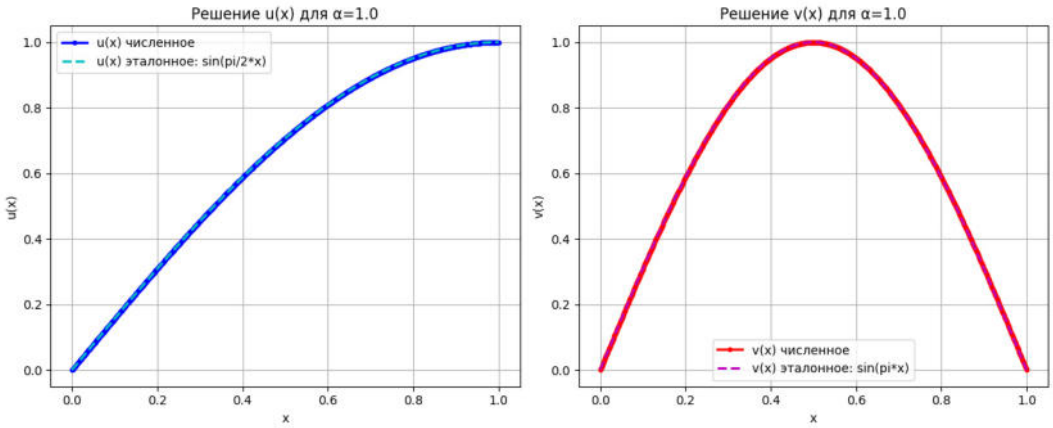

Рис. 3: $N = 1000$

Пример 4 Убедимся в непрерывности решения в зависимости от параметра α. Возьмем $f(x) = cosx$, $g(x) = sinx$

Погрешность: 0.017

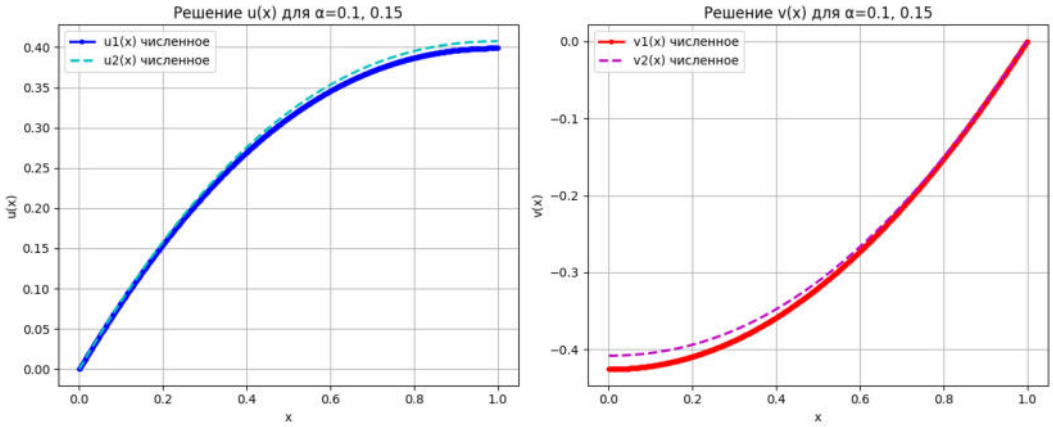

Рис. 4: $N = 1000$

Пример 5 Рассмотрим влияния нелинейного члена в зависимости от параметра α. Возьмем $f(x) = cosx$, $g(x) = sinx$

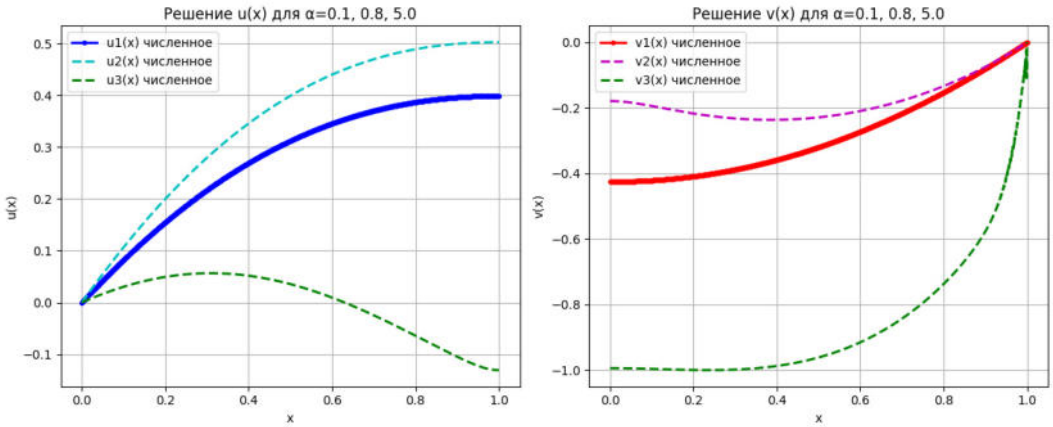

Рис. 5: $N = 1000$

5 Физическая интерпретация

- Уравнение для $u(x)$ описывает колебания струны под действием внешней силы $f(x)$ с дополнительным нелинейным членом, учитывающим взаимодействие с полем $v(x)$
- Уравнение для $v(x)$ моделирует перенос некоторой величины (например, тепла или массы) с источником $g(x)$
- Краевые условия $u(0) = 0$, $u'(1) = 0$ соответствуют закрепленному левому концу и свободному правому концу струны
- Условие $v(1) = 0$ означает отсутствие потока на правой границе

6 Выводы

В результате проведенного исследования получены следующие выводы:
1. Реализован численый алгоритм решения на Python.
2. Рассмотрен тривиальный случай в примере 1.
3. Проверена непрерывная зависимость решения от параметра α.
4. Получили точные численные решения при известном точном решении.
5. Посмотрели влияние параметра α на полученное решение.

7 Список литературы

1. *Богачев К. Ю.* Методы решения линейных систем и нахождения собственных значений // Труды механико-математического факультета МГУ. — М.: Изд-во мехмата МГУ, 1998. — 256 с.

2. *Ортега Дж., Пул У.* Введение в численные методы решения дифференциальных уравнений. — М.: Наука, 1986. — 320 с.

8 Приложение. Код программы.

```python
import numpy as np
import matplotlib.pyplot as plt

class NonlinearSystemSolver:
    def __init__(self, N, alpha, f, g):
        self.N = N
        self.h = 1.0 / N
        self.alpha = alpha
        self.f = f
        self.g = g
        self.x = np.linspace(0, 1, N + 1)
        self.num_unknowns = 2 * N

    def _Q(self, u, v):
        denominator = u**2 + v**2 + 1e-8
        return self.alpha * u * v / denominator

    def _dQ_du(self, u, v):
        denominator = u**2 + v**2 + 1e-8

        return self.alpha * v * (v**2 + 1e-8 - u**2) / (denominator**2)

    def _dQ_dv(self, u, v):
        denominator = u**2 + v**2 + 1e-8

        return self.alpha * u * (u**2 + 1e-8 - v**2) / (denominator**2)

    def build_F_vector(self, u_vec, v_vec):

        F = np.zeros(self.num_unknowns)


        for i in range(1, self.N):
            term_u_xx = u_vec[i-1] - 2*u_vec[i] + u_vec[i+1]
            nonlin_term = self.h**2 * self._Q(u_vec[i], v_vec[i])
            f_term = self.h**2 * self.f(self.x[i])
            F[i-1] = term_u_xx - nonlin_term + f_term

        F[self.N-1] = 3*u_vec[self.N] - 4*u_vec[self.N-1] + u_vec[self.N-2]


        F[self.N] = -3*v_vec[0] + 4*v_vec[1] - v_vec[2] - 2*self.h*self.g(self.x[0])


        for i in range(1, self.N):
            term_v_x = v_vec[i+1] - v_vec[i-1]
            nonlin_term = 2*self.h * self._Q(u_vec[i], v_vec[i])
            g_term = 2*self.h * self.g(self.x[i])
            F[self.N + i] = term_v_x - nonlin_term - g_term

        return F

    def build_jacobian_matrix(self, u_vec, v_vec):

        J = np.zeros((self.num_unknowns, self.num_unknowns))


```

```
        for i in range(1, self.N):
            row = i - 1
            if i > 1: J[row, i-2] = 1 #      u_{i-1}
            J[row, i-1] = -2 - self.h**2 * self._dQ_du(u_vec[i], v_vec[i]) # u_i
            J[row, i] = 1 #      u_{i+1}


        row = self.N - 1
        J[row, self.N-3] = 1   #      u_{N-2}
        J[row, self.N-2] = -4  #      u_{N-1}
        J[row, self.N-1] = 3   #      u_N


        for i in range(1, self.N):
            row = i - 1
            col = self.N + i #                        v_i
            J[row, col] = -self.h**2 * self._dQ_dv(u_vec[i], v_vec[i])


        for i in range(1, self.N):
            row = self.N + i
            col = i-1 #                        u_i (  .  . u_1 -  )
            J[row, col] = -2 * self.h * self._dQ_du(u_vec[i], v_vec[i])


        row = self.N
        J[row, self.N] = -3    #      v_0
        J[row, self.N+1] = 4     #      v_1
        J[row, self.N+2] = -1    #      v_2


        for i in range(1, self.N):
            row = self.N + i
            J[row, self.N + i - 1] = -1 #      v_{i-1}
            J[row, self.N + i] = -2*self.h * self._dQ_dv(u_vec[i], v_vec[i]) # v_i
            if i < self.N - 1:
                J[row, self.N + i + 1] = 1 #      v_{i+1}

        return J

    def solve(self, max_iter=100, tol=1e-8, verbose=True):
        if abs(self.alpha) < 1e-12:
            if verbose: print("              alpha=0,                          .")
            return self.solve_linear_case()

        u_lin, v_lin = self.solve_linear_case()
        X = np.concatenate([u_lin[1:], v_lin[:-1]])

        if verbose: print("              ...")
        for iteration in range(max_iter):
            u = np.zeros(self.N + 1)
            v = np.zeros(self.N + 1)

            u[1:] = X[:self.N]
            v[:-1] = X[self.N:]
```

```
            F = self.build_F_vector(u, v)
            J = self.build_jacobian_matrix(u, v)

            try:
                delta_X = np.linalg.solve(J, -F)
            except np.linalg.LinAlgError:
                if verbose: print("            :
                    .                                                    .")
                return u, v #

            X += delta_X
            norm = np.linalg.norm(delta_X)
            if verbose: print(f"                    {iteration+1}:
                    = {norm:.2e}")

            if norm < tol:
                if verbose: print(f"
                        {iteration+1}")
                break
        else:
             if verbose: print("
                                ,
 .")

        u_final = np.zeros(self.N + 1)
        v_final = np.zeros(self.N + 1)
        u_final[1:] = X[:self.N]
        v_final[:-1] = X[self.N:]

        return u_final, v_final

    def solve_linear_case(self):
        u = np.zeros(self.N+1)
        A_u = np.zeros((self.N, self.N))
        b_u = np.zeros(self.N)

        for i in range(1, self.N):
            row = i - 1
            if i > 1: A_u[row, i-2] = 1
            A_u[row, i-1] = -2
            A_u[row, i] = 1
            b_u[row] = -self.h**2 * self.f(self.x[i])

        A_u[self.N-1, self.N-3] = 1
        A_u[self.N-1, self.N-2] = -4
        A_u[self.N-1, self.N-1] = 3
        b_u[self.N-1] = 0

        u_part = np.linalg.solve(A_u, b_u)
        u[1:] = u_part

        v = np.zeros(self.N+1)

        v[self.N] = 0  # v(1) = 0
        for i in range(self.N-1, -1, -1):

            integral = 0.0
            for j in range(i, self.N):
```

```
                integral += (self.g(self.x[j]) + self.g(self.x[j+1])) * self.h / 2
            v[i] = -integral

        return u, v

    def calculate_residuals_norm(self, u, v):
        F = self.build_F_vector(u, v)
        return np.linalg.norm(F)

    def calculate_exact_solution_error(self, u_num1, v_num1, u_num2, v_num2):

        error_u = np.sqrt(self.h * np.sum((u_num1 - u_num2)**2))
        error_v = np.sqrt(self.h * np.sum((v_num1 - v_num2)**2))

        max_error_u = np.max(np.abs(u_num1 - u_num2))
        max_error_v = np.max(np.abs(v_num1 - v_num2))

        return max(max_error_u, max_error_v)

    def visualize(self, u1, v1, u2, v2, u3, v3, title_suffix=""):
        plt.figure(figsize=(12, 5))


        plt.subplot(1, 2, 1)
        plt.plot(self.x, u1, 'b-o', linewidth=2, markersize=3, label='u1(x)')
        plt.plot(self.x, u2, 'c--', linewidth=2, label='u2(x)')
        plt.plot(self.x, u3, 'g--', linewidth=2, label='u3(x)')
        plt.xlabel('x')
        plt.ylabel('u(x)')
        plt.title(f'                u(x) {title_suffix}')
        plt.grid(True)
        plt.legend()

        plt.subplot(1, 2, 2)
        plt.plot(self.x, v1, 'r-o', linewidth=2, markersize=3, label='v1(x)')
        plt.plot(self.x, v2, 'm--', linewidth=2, label='v2(x)')
        plt.plot(self.x, v3, 'g--', linewidth=2, label='v3(x)')
        plt.xlabel('x')
        plt.ylabel('v(x)')
        plt.title(f'                v(x) {title_suffix}')
        plt.grid(True)
        plt.legend()

        plt.tight_layout()
        plt.show()
```